

GERMANCREDIT CLASIFICACION

Acerca del conjunto de datos

El conjunto de datos original contiene 1000 entradas con 20 atributos categóricos/simbólicos, preparado por el profesor Hofmann. En este conjunto de datos, cada entrada representa a una persona que solicita un crédito bancario. Cada persona se clasifica como de riesgo crediticio alto o bajo según el conjunto de atributos. El enlace al conjunto de datos original se encuentra a continuación.

Contenido

- Edad (numérico)
- Sexo (texto: masculino, femenino)
- Trabajo (numérico: 0 - no cualificado y no residente, 1 - no cualificado y residente, 2 - cualificado, 3 - altamente cualificado)
- Cuenta de ahorros: (pequeña, moderada, muy buena, buena, nodata)
- Vivienda (texto: propia, de alquiler o gratuita) Cuentas de ahorro (texto: poco, moderado, bastante rico, rico)
- Cuenta corriente (numérico, en marcos alemanes)
- Importe del crédito (numérico, en marcos alemanes)
- Duración (numérico, en meses)
- Propósito (texto: coche, muebles/equipo, radio/TV, electrodomésticos, reparaciones, educación, negocios, vacaciones/otros)

OBJETIVO

Evaluar modelos de clasificación para predicción de riesgo de nuevos clientes

LIBRERIAS

```
In [1]: # Manipulación de datos
import numpy as np
import pandas as pd

#Graficos
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from plotly import tools

#Preparación de datos
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder, OrdinalEncoder, LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn import set_config
from sklearn.impute import SimpleImputer
from sklearn.metrics import classification_report

#Modelado
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn import tree
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.gaussian_process.kernels import RBF
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import cross_val_score

#Evaluación
from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve

In [2]: df = pd.read_excel('german_credit_data.xlsx', index_col=0)
df.shape
df.head()
```

```
Out[2]:
```

	edad	sexo	trabajos	vivienda	cuenta ahorros	cuenta cheques	monto credito	duracion	proposito	rango_edad	rango_credito	riesgo
id												
915	32	femenino	3	propia	pequeña	moderada	18424	48	vacaciones	Joven	muy_alto	riesgoso
95	58	masculino	2	arriendo	pequeña	moderada	15945	54	negocio	Mayor	muy_alto	riesgoso
818	43	masculino	3	propia	pequeña	pequeña	15857	36	vacaciones	Adulto	muy_alto	riesgoso
887	23	masculino	2	propia	pequeña	moderada	15672	48	negocio	Adolescente	muy_alto	riesgoso
637	21	masculino	2	propia	pequeña	no data	15653	60	radio/TV	Adolescente	muy_alto	riesgoso

```
In [3]: df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 1000 entries, 915 to 725
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   edad                   1000 non-null  int64
1   sexo                   1000 non-null  object
2   trabajos               1000 non-null  int64
3   vivienda               1000 non-null  object
4   cuenta ahorros         1000 non-null  object
5   cuenta cheques         1000 non-null  object
6   monto credito          1000 non-null  int64
7   duracion               1000 non-null  int64
8   proposito              1000 non-null  object
9   rango_edad             1000 non-null  object
10  rango_credito          1000 non-null  object
11  riesgo                 1000 non-null  object
dtypes: int64(4), object(8)
memory usage: 101.6+ KB
```

LIMPIEZA

```
In [4]: ### Eliminacion de duplicados
df.duplicated().sum()

### Eliminacion de nulos
df.isnull().sum()
```

```
Out[4]: edad          0
sexo            0
trabajos        0
vivienda        0
cuenta ahorros  0
cuenta cheques  0
monto credito   0
duracion        0
proposito       0
rango_edad      0
rango_credito   0
riesgo          0
dtype: int64
```

EDA VARIABLES CATEGORICAS

```
In [5]: categoricas = df.select_dtypes(include=["object"]).columns.tolist()

cols = 3
rows = (len(categoricas) + cols - 1) // cols

# Crear subplots
fig = make_subplots(rows=rows, cols=cols, subplot_titles=categoricas)

# Agregar un gráfico de barras por cada variable categórica
for i, var in enumerate(categoricas):
    row = i // cols + 1
    col = i % cols + 1

    # Contar frecuencias de cada categoría y ordenar de forma descendente
    value_counts = df[var].value_counts().sort_values(ascending=False)

    fig.add_trace(
        go.Bar(x=value_counts.index, y=value_counts.values, name=var, marker_color="#6a32d2"),
        row=row, col=col
    )
```

```
# Ajustes de presentación
fig.update_layout(
    height=300 * rows,
    width=1400,
    title_text='Variables categóricas',
    template='plotly',
    showlegend=False
)

fig.show()
```

Categoricas por monto de crédito

```
In [6]: ### Variables categoricas por monto

import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Asumiendo que ya tienes un DataFrame llamado df con una columna 'monto'
categoricas = df.select_dtypes(include=["object"]).columns.tolist()

cols = 2
rows = (len(categoricas) + cols - 1) // cols

# Crear subplots
fig = make_subplots(rows=rows, cols=cols, subplot_titles=categoricas)

# Agregar un gráfico de barras por cada variable categórica relacionado con el monto
for i, var in enumerate(categoricas):
    row = i // cols + 1
    col = i % cols + 1

    # Calcular el monto credito por cada categoría
    monto_por_categoria = df.groupby(var)['monto credito'].mean().sort_values(ascending=False)

    fig.add_trace(
        go.Bar(
            x=monto_por_categoria.index,
            y=monto_por_categoria.values,
            name=var,
            marker_color="#cd1616",
            text=monto_por_categoria.values.round(2), # Mostrar valores en las barras
            textposition='auto'
        ),
        row=row, col=col
    )

    # Actualizar títulos de los ejes para cada subplot
    fig.update_xaxes(title_text=var, row=row, col=col)
    fig.update_yaxes(title_text="monto credito", row=row, col=col)

# Ajustes de presentación
fig.update_layout(
    height=350 * rows, # Un poco más alto para acomodar las etiquetas
    width=1400,
    title_text='monto credito por variables categóricas',
    template='plotly',
    showlegend=False
)

fig.show()
```

Categoricas por riesgo

```
In [7]: # Graficar barras de variables categóricas segmentadas por 'riesgo'

cols = 3
rows = (len(categoricas) + cols - 1) // cols

fig = make_subplots(rows=rows, cols=cols, subplot_titles=categoricas)

for i, var in enumerate(categoricas):
    row = i // cols + 1
    col = i % cols + 1
    # Agrupar y contar por categoría y riesgo
    counts = df.groupby([var, 'riesgo']).size().unstack(fill_value=0)
    # Ordenar por el total de cada categoría
    counts = counts.loc[counts.sum(axis=1).sort_values(ascending=False).index]
    for riesgo_value in counts.columns:
        fig.add_trace(
```

```

        go.Bar(
            x=counts.index,
            y=counts[riesgo_value],
            name=f"{var} - {riesgo_value}",
            marker_color="#ffca5" if riesgo_value == "no_riesgo" else "#fc6f6f",
            showlegend=(i == 0)
        ),
        row=row, col=col
    )

fig.update_layout(
    height=300 * rows,
    width=1400,
    title_text='Variables categóricas por riesgo',
    template='plotly',
    barmode='stack'
)
fig.show()

```

EDA VARIABLES NUMERICAS

In [8]: `df.describe()`

Out[8]:

	edad	trabajos	monto credito	duracion
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	35.546000	1.904000	3271.258000	20.903000
std	11.375469	0.653614	2822.736876	12.058814
min	19.000000	0.000000	250.000000	4.000000
25%	27.000000	2.000000	1365.500000	12.000000
50%	33.000000	2.000000	2319.500000	18.000000
75%	42.000000	2.000000	3972.250000	24.000000
max	75.000000	3.000000	18424.000000	72.000000

HISTOGRAMA

In [9]: `numericas = df.select_dtypes(include=["int", "float"]).columns.tolist()`

```

# Número de columnas por fila (ajústalo según cuántas variables tengas)
cols = 3
rows = (len(numericas) + cols - 1) // cols # redondeo hacia arriba

# Crear subplots
fig = make_subplots(rows=rows, cols=cols, subplot_titles=numericas)

# Agregar un histograma por variable
for i, var in enumerate(numericas):
    row = i // cols + 1
    col = i % cols + 1
    fig.add_trace(
        go.Histogram(x=df[var], name=var, marker_color="#b836e8", nbinsx=30),
        row=row, col=col
    )

# Ajustes de presentación
fig.update_layout(
    height=300 * rows,
    width=1400,
    title_text='Distribuciones de Variables Numéricas',
    template='plotly_white',
    showlegend=False
)

fig.show()

```

BOX-PLOT

In [10]: `# Número de columnas por fila (ajústalo según cuántas variables tengas)`

```

cols = 2
rows = (len(numericas) + cols - 1) // cols # redondeo hacia arriba

# Crear subplots
fig = make_subplots(rows=rows, cols=cols, subplot_titles=numericas)

```

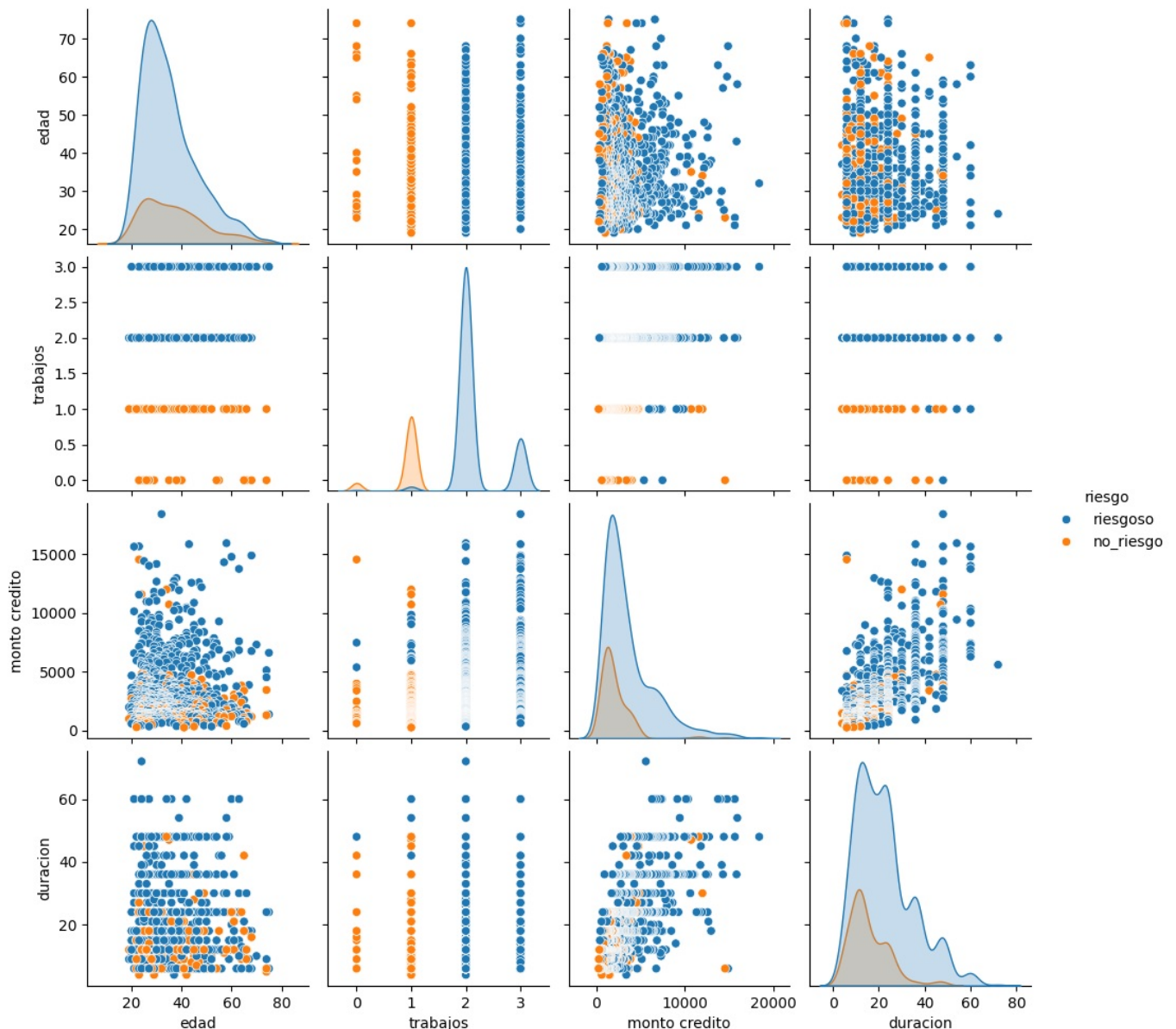
```
# Agregar un histograma por variable
for i, var in enumerate(numericas):
    row = i // cols + 1
    col = i % cols + 1
    fig.add_trace(
        go.Box(x=df[var], name=var, marker_color="#d13e3e"),
        row=row, col=col
    )

# Actualizar diseño
fig.update_layout(height=800, width=1300, title_text="Histogramas sin Outliers (Método IQR)",
                  showlegend=False)
fig.update_xaxes(title_text="Valor")
fig.update_yaxes(title_text="Frecuencia")

fig.show()
```

GRAFICO DE DISPERSION

```
In [11]: # identificación de grafica de dispersión con la variable objetivo
sns.pairplot(data=df, hue='riesgo', kind = 'scatter')
plt.show()
```



REALIZAR UNA EJECUCION SOBRE ESTA LINEA PARA LOS DIFERENTES ENCODERS

TRANSFORMACION DE VARIABLES A NUMERICOS

```
In [415]: # =====
# ORDENAR LAS VARIABLES PARA TRANSFORMARLAS
# =====
```

```

# Variable objetivo
target = 'riesgo'

# ① Columnas numéricas (solo verdaderas numéricas continuas o discretas)
numCols = df.select_dtypes(include=['int64', 'float64']).columns.to_list()

# Eliminar la variable objetivo si aparece por error
if target in numCols:
    numCols.remove(target)

# ② Columnas categóricas binarias (solo 2 valores)
# Ejemplo: 'sexo' = {'male', 'female'} o {'M', 'F'}
binCols = ['sexo']

# ③ Columnas categóricas nominales (sin orden natural)
# ⚠ Si alguna de las siguientes NO tiene un orden definido (por ejemplo, 'proposito'),
# la pasamos aquí en lugar de las ordinales.
catCols = ['proposito']

# ④ Columnas categóricas ordinales (con un orden lógico o jerárquico)
ordCols = [
    'cuenta ahorros',
    'vivienda',
    'cuenta cheques',
    'rango_edad',
    'rango_credito'
]

# =====
# Verificación rápida
# =====
print("Numéricas:", numCols)
print("Binarias:", binCols)
print("Nominales:", catCols)
print("Ordinales:", ordCols)
print("Objetivo:", target)

```

```

Numéricas: ['edad', 'trabajos', 'monto credito', 'duracion']
Binarias: ['sexo']
Nominales: ['proposito']
Ordinales: ['cuenta ahorros', 'vivienda', 'cuenta cheques', 'rango_edad', 'rango_credito']
Objetivo: riesgo

```

Valores unicos de las variables categoricas

In [416..

```

# =====
# MOSTRAR VALORES ÚNICOS DE VARIABLES CATEGÓRICAS
# =====

import pandas as pd

# Lista de columnas categóricas y ordinales (ajústala según tu dataset)
categorical_cols = [
    'cuenta ahorros',
    'vivienda',
    'cuenta cheques',
    'proposito',
    'rango_edad',
    'rango_credito'
]

# Mostrar valores únicos y su frecuencia relativa
for col in categorical_cols:
    print(f"\n --- {col.upper()} ---")
    print(df[col].value_counts(dropna=False, normalize=True).apply(lambda x: f"{x*100:.1f}%"))
    print("Valores únicos:", df[col].unique())

```

```

--- CUENTA AHORROS ---
cuenta ahorros
pequeña      60.3%
no data      18.3%
moderada     10.3%
muy buena    6.3%
buena        4.8%
Name: proportion, dtype: object
Valores únicos: ['pequeña' 'moderada' 'no data' 'muy buena' 'buena']

--- VIVIENDA ---
vivienda
propia       71.3%
arriendo     17.9%
libre        10.8%
Name: proportion, dtype: object
Valores únicos: ['propia' 'arriendo' 'libre']

--- CUENTA CHEQUES ---
cuenta cheques
no data      39.4%
pequeña      27.4%
moderada     26.9%
buena        6.3%
Name: proportion, dtype: object
Valores únicos: ['moderada' 'pequeña' 'no data' 'buena']

--- PROPOSITO ---
proposito
automovil    33.7%
radio/TV     28.0%
equipo       18.1%
negocio      9.7%
educacion    5.9%
reparaciones 2.2%
vacaciones   1.2%
electrodomesticos 1.2%
Name: proportion, dtype: object
Valores únicos: ['vacaciones' 'negocio' 'radio/TV' 'automovil' 'equipo' 'educacion'
'reparaciones' 'electrodomesticos']

--- RANGO_EDAD ---
rango_edad
Joven        39.8%
Adulto       29.9%
Adolescente  19.0%
Mayor        11.3%
Name: proportion, dtype: object
Valores únicos: ['Joven' 'Mayor' 'Adulto' 'Adolescente']

--- RANGO_CREDITO ---
rango_credito
bajo         69.6%
medio        14.8%
sin_credito  11.6%
alto         3.5%
muy_alto     0.5%
Name: proportion, dtype: object
Valores únicos: ['muy_alto' 'alto' 'medio' 'bajo' 'sin_credito']

```

Ordenar las variables categoricas

```

In [417_: # =====
# VARIABLES POR TIPO
# =====

# Variable objetivo
target = 'riesgo'

# Numéricas (ajusta según tu df real)
numCols = df.select_dtypes(include=['int64', 'float64']).columns.to_list()
if target in numCols:
    numCols.remove(target)

# Binarias
binCols = ['sexo']

# Nominales (sin orden natural)
catCols = ['proposito']

# Ordinales (con orden definido)
ordCols = ['cuenta ahorros', 'vivienda', 'cuenta cheques', 'rango_edad', 'rango_credito']

```

```
# Mapeo de orden por variable
ord_mappings = {
    'cuenta ahorros': ['no data', 'pequeña', 'moderada', 'buena', 'muy buena'],
    'vivienda': ['arriendo', 'libre', 'propia'],
    'cuenta cheques': ['no data', 'pequeña', 'moderada', 'buena'],
    'rango_edad': ['Adolescente', 'Joven', 'Adulto', 'Mayor'],
    'rango_credito': ['sin_credito', 'bajo', 'medio', 'alto', 'muy_alto']
}

# Crear lista de categorías en el orden que espera OrdinalEncoder
ord_categories = [ord_mappings[col] for col in ordCols]
```

Encoders

```
In [418.. # =====
# 2 PIPELINES POR TIPO DE VARIABLE
# =====

# Numéricas → imputación + escalado
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Binarias → imputación + codificación ordinal (0/1)
bin_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder())
])

# Nominales → imputación + one-hot encoding
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

# Ordinales → imputación + codificación ordinal con orden definido
ord_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(categories=ord_categories))
])

# =====
# 3 UNIR TODOS LOS PIPELINES EN UN COLUMNTRANSFORMER
# =====

preprocessor = ColumnTransformer(transformers=[
    ('num', num_pipeline, numCols),
    ('bin', bin_pipeline, binCols),
    ('cat', cat_pipeline, catCols),
    ('ord', ord_pipeline, ordCols)
])
```

Evaluación de modelos con el encoders nuevo

```
In [419.. # =====
# 4 CREAR EL PIPELINE FINAL CON UN MODELO
# =====

model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight='balanced',
    n_jobs=-1
)

pipe = Pipeline([
    ('preproc', preprocessor),
    ('model', model)
])

# =====
# 5 DIVISIÓN DEL CONJUNTO DE DATOS Y ENTRENAMIENTO
# =====

X = df.drop(columns=[target])
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
```

```

)

pipe.fit(X_train, y_train)

# =====
# 6 EVALUACIÓN DEL MODELO
# =====

y_pred = pipe.predict(X_test)

print(" CLASSIFICATION REPORT")
print(classification_report(y_test, y_pred))

print("\n MATRIZ DE CONFUSIÓN")
print(confusion_matrix(y_test, y_pred))

# =====
# 7 VER NOMBRES DE FEATURES TRANSFORMADOS (opcional)
# =====

feature_names = pipe.named_steps['preproc'].get_feature_names_out()
print("\nNombres de columnas después del preprocesamiento:")
print(feature_names)

```

```

CLASSIFICATION REPORT
              precision    recall  f1-score   support

   no_riesgo         1.00        0.98        0.99         42
   riesgoso          0.99        1.00        1.00        158

 accuracy                   0.99         200
 macro avg          1.00        0.99        0.99         200
weighted avg          1.00        0.99        0.99         200

```

```

MATRIZ DE CONFUSIÓN
[[ 41   1]
 [  0 158]]

```

```

Nombres de columnas después del preprocesamiento:
['num_edad' 'num_trabajos' 'num_monto_credito' 'num_duracion'
 'bin_sexo' 'cat_proposito_automovil' 'cat_proposito_educacion'
 'cat_proposito_electrodomesticos' 'cat_proposito_equipo'
 'cat_proposito_negocio' 'cat_proposito_radio/TV'
 'cat_proposito_reparaciones' 'cat_proposito_vacaciones'
 'ord_cuenta_ahorros' 'ord_vivienda' 'ord_cuenta_cheques'
 'ord_rango_edad' 'ord_rango_credito']

```

Comparacion de modelos con el nuevo encoder

In [420]

```

# =====
# COMPARACIÓN DE MODELOS - CORRECCIÓN COMPLETA
# =====

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import cross_validate, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPClassifier
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# =====
# 1 CODIFICAR VARIABLE OBJETIVO A BINARIA
# =====
# (asegúrate de haber definido X, y previamente)

le = LabelEncoder()
y_encoded = le.fit_transform(y)

print("Clases codificadas:", dict(zip(le.classes_, le.transform(le.classes_))))

# =====
# 2 DEFINIR MODELOS A COMPARAR
# =====

models = {
    'Logistic Regression': LogisticRegression(max_iter=2000, class_weight='balanced'),

```

```

'SVM': SVC(kernel='rbf', probability=True, class_weight='balanced'),
'KNN': KNeighborsClassifier(),
'Decision Tree': DecisionTreeClassifier(random_state=42, class_weight='balanced'),
'Random Forest': RandomForestClassifier(n_estimators=200, random_state=42, class_weight='balanced', n_jobs=
'Gradient Boosting': GradientBoostingClassifier(random_state=42),
'Naive Bayes': GaussianNB(),
'MLP Neural Net': MLPClassifier(max_iter=500, random_state=42)
}

# =====
# 3 CONFIGURACIÓN DE VALIDACIÓN CRUZADA
# =====
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = {
    'accuracy': 'accuracy',
    'precision': 'precision_macro',
    'recall': 'recall_macro',
    'f1': 'f1_macro'
}

# =====
# 4 EVALUAR CADA MODELO
# =====

results = []

for name, model in models.items():
    pipe_model = Pipeline([
        ('preproc', preprocessor),
        ('model', model)
    ])

    scores = cross_validate(pipe_model, X, y_encoded, cv=cv, scoring=scoring, n_jobs=-1)

    results.append({
        'Modelo': name,
        'Accuracy': np.mean(scores['test_accuracy']),
        'Precision': np.mean(scores['test_precision']),
        'Recall': np.mean(scores['test_recall']),
        'F1': np.mean(scores['test_f1'])
    })

df_results = pd.DataFrame(results).sort_values(by='F1', ascending=False).reset_index(drop=True)

# =====
# 5 MOSTRAR RESULTADOS
# =====

print("\n RESULTADOS PROMEDIO DE VALIDACIÓN CRUZADA (5 folds):")
print(df_results.round(3))

# =====
# 6 GRAFICAR COMPARACIÓN
# =====

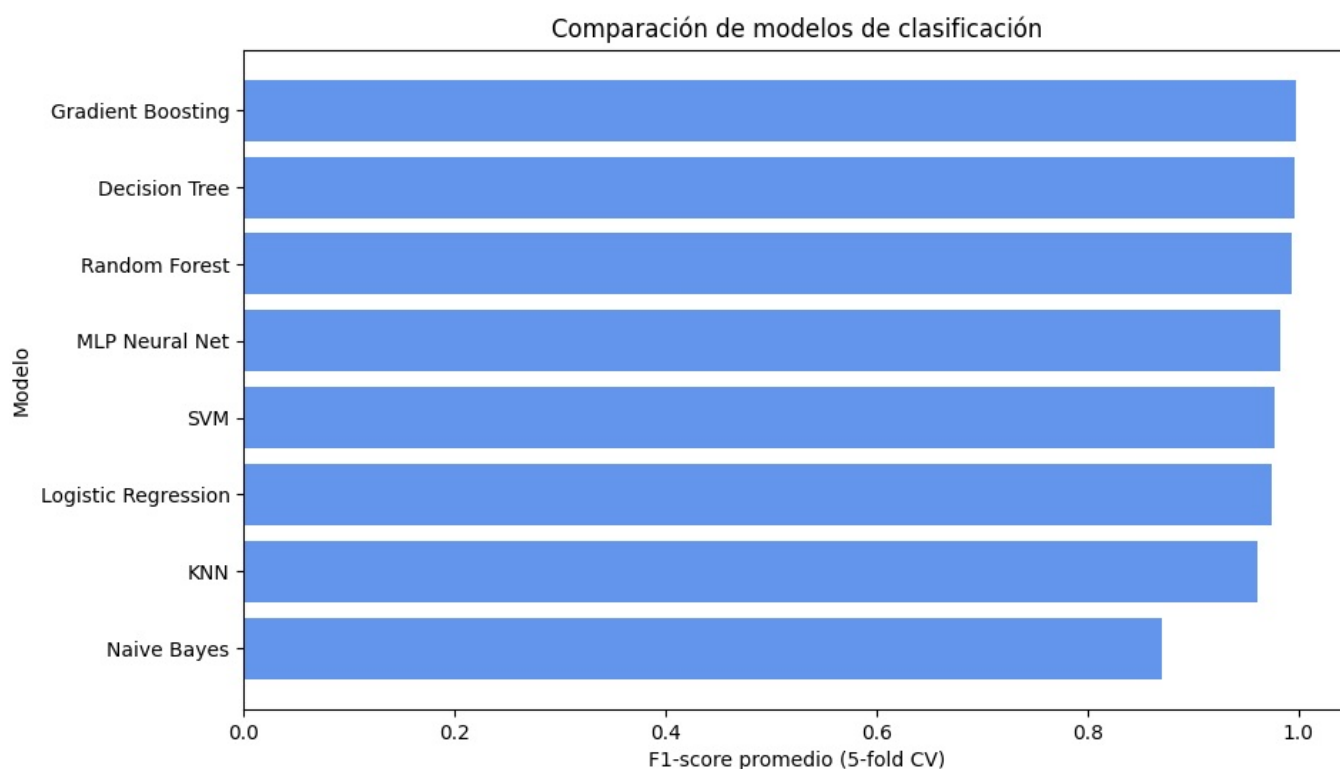
plt.figure(figsize=(10,6))
plt.barh(df_results['Modelo'], df_results['F1'], color='cornflowerblue')
plt.xlabel('F1-score promedio (5-fold CV)')
plt.ylabel('Modelo')
plt.title('Comparación de modelos de clasificación')
plt.gca().invert_yaxis()
plt.show()

```

Clases codificadas: {'no_riesgo': np.int64(0), 'riesgoso': np.int64(1)}

RESULTADOS PROMEDIO DE VALIDACIÓN CRUZADA (5 folds):

	Modelo	Accuracy	Precision	Recall	F1
0	Gradient Boosting	0.999	0.999	0.998	0.998
1	Decision Tree	0.998	0.997	0.997	0.997
2	Random Forest	0.996	0.997	0.990	0.994
3	MLP Neural Net	0.989	0.981	0.986	0.983
4	SVM	0.985	0.968	0.989	0.978
5	Logistic Regression	0.983	0.966	0.984	0.975
6	KNN	0.975	0.975	0.949	0.961
7	Naive Bayes	0.882	0.896	0.910	0.871



HASTA AQUÍ OCURRE UN LABEL ENCODER DE VARIABLES DIFERENTE

```
In [19]: # viejo código de transformación de variables

# ORDENAR LAS VARIABLES PARA TRANSFORMARLAS
# Columnas numéricas
numCols = df.select_dtypes(include=['int64']).columns.to_list()

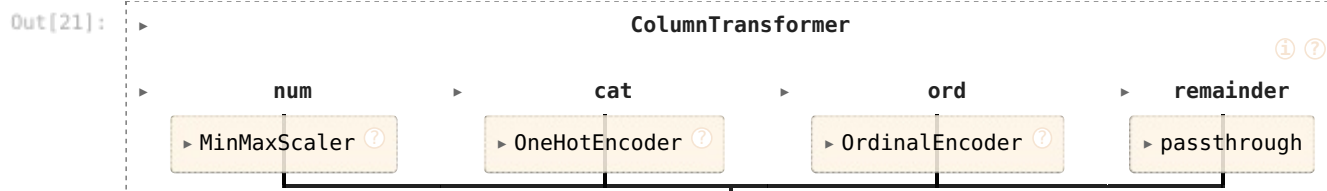
# Columnas categóricas con más de 2 valores (similar a ordCols, pero no necesariamente ordinales)
catCols = ['sexo']

# Ordinales
ordCols = ['riesgo', 'cuenta ahorros', 'vivienda', 'cuenta cheques', 'proposito', 'rango_edad', 'rango_credito']
```

```
In [20]: #TRANSFORMACIÓN DE VARIABLES
numTransformer = Pipeline(steps=[('scaler', MinMaxScaler())])
catTransformer = Pipeline(steps=[('onehot', OneHotEncoder(handle_unknown='ignore'))])
ordTransformer = Pipeline(steps=[('ordinal', OrdinalEncoder(handle_unknown='error'))])
```

```
In [21]: preprocesador = ColumnTransformer(transformers = [
    ('num', numTransformer, numCols),
    ('cat', catTransformer, catCols),
    ('ord', ordTransformer, ordCols)
], remainder = 'passthrough')

set_config(display = 'diagram')
preprocesador
```



```
In [22]: datosPre = preprocesador.fit_transform(df)
datosPre
```

```
Out[22]: array([[2.32142857e-01, 1.00000000e+00, 1.00000000e+00, ...,
              7.00000000e+00, 2.00000000e+00, 3.00000000e+00],
              [6.96428571e-01, 6.66666667e-01, 8.63596346e-01, ...,
              4.00000000e+00, 3.00000000e+00, 3.00000000e+00],
              [4.28571429e-01, 1.00000000e+00, 8.58754264e-01, ...,
              7.00000000e+00, 1.00000000e+00, 3.00000000e+00],
              ...,
              [5.89285714e-01, 6.66666667e-01, 4.84208210e-03, ...,
              5.00000000e+00, 3.00000000e+00, 4.00000000e+00],
              [5.35714286e-02, 3.33333333e-01, 1.43061516e-03, ...,
              0.00000000e+00, 0.00000000e+00, 4.00000000e+00],
              [3.92857143e-01, 3.33333333e-01, 0.00000000e+00, ...,
              0.00000000e+00, 1.00000000e+00, 4.00000000e+00]], shape=(1000, 13))
```

```
In [23]: # colocarle el nombre a las columnas
codBin = preprocesador.named_transformers_['cat']['onehot'].get_feature_names_out(catCols)
labels = np.concatenate([numCols, codBin, ordCols])
labels
```

```
Out[23]: array(['edad', 'trabajos', 'monto credito', 'duracion', 'sexo_femenino',
              'sexo_masculino', 'riesgo', 'cuenta ahorros', 'vivienda',
              'cuenta cheques', 'proposito', 'rango_edad', 'rango_credito'],
              dtype=object)
```

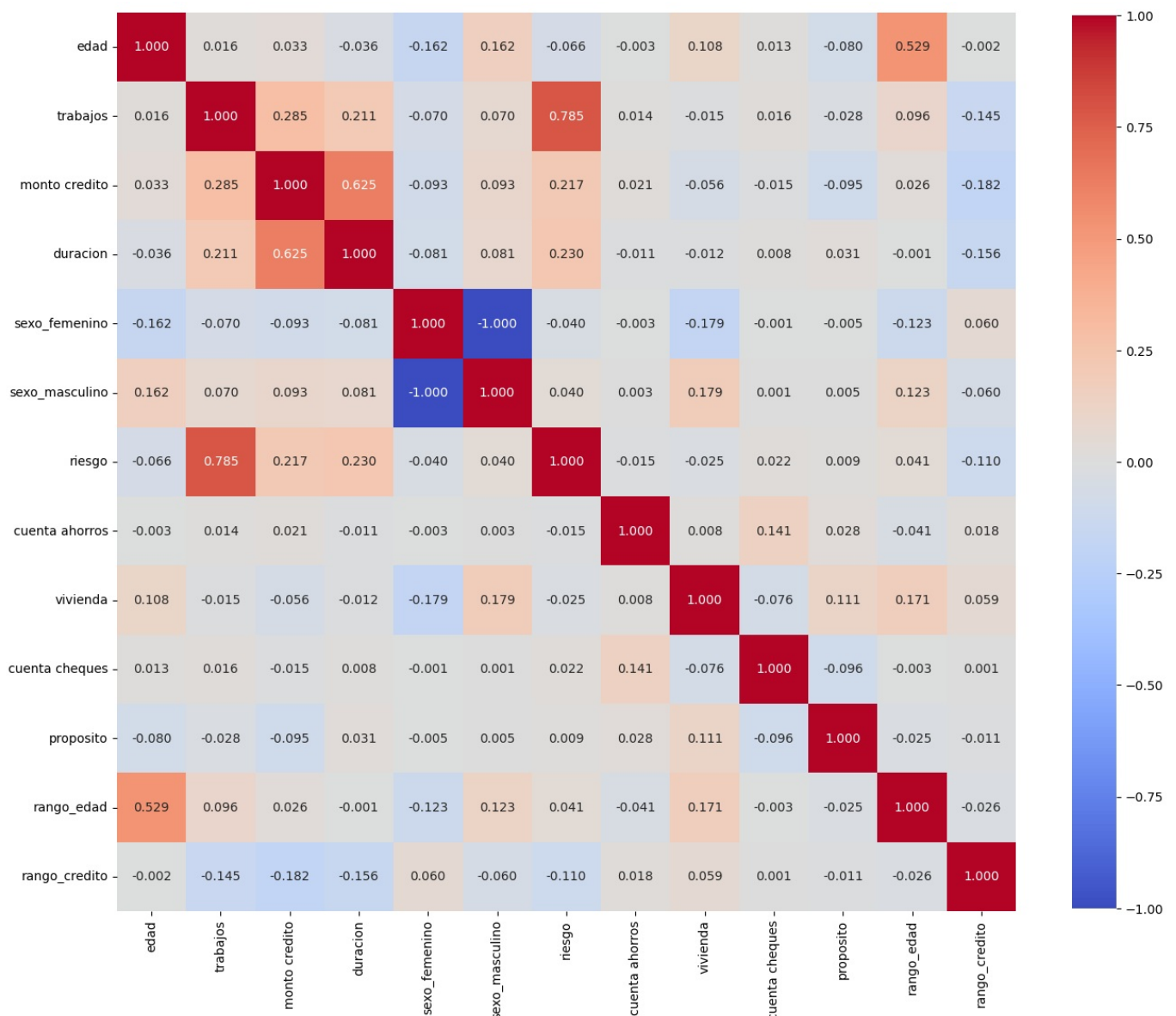
```
In [25]: # convertir a dataframe
datosPro = pd.DataFrame(datosPre, columns=labels)
datosPro.head()
```

Out[25]:

	edad	trabajos	monto credito	duracion	sexo_femenino	sexo_masculino	riesgo	cuenta ahorros	vivienda	cuenta cheques	proposito	rango_
0	0.232143	1.000000	1.000000	0.647059	1.0	0.0	1.0	4.0	2.0	1.0	7.0	
1	0.696429	0.666667	0.863596	0.735294	0.0	1.0	1.0	4.0	0.0	1.0	4.0	
2	0.428571	1.000000	0.858754	0.470588	0.0	1.0	1.0	4.0	2.0	3.0	7.0	
3	0.071429	0.666667	0.848575	0.647059	0.0	1.0	1.0	4.0	2.0	1.0	4.0	
4	0.035714	0.666667	0.847529	0.823529	0.0	1.0	1.0	4.0	2.0	2.0	5.0	

HEADMAP

```
In [26]: plt.figure(figsize = (16, 16))
sns.heatmap(datosPro.corr(), annot = True, fmt = '.3f', cmap = 'coolwarm', square = True, cbar_kws = {'shrink':
plt.show()
```



ENTRENAMIENTO

```
In [27]: X = datosPro.drop('riesgo', axis = 1)
y = datosPro['riesgo']
```

```
In [28]: X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, stratify=y, random_state=42)
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

```
Out[28]: ((800, 12), (200, 12), (800,), (200,))
```

MODELOS

```
In [29]: Dict_classifiers = {
    "Logistic Regression": LogisticRegression(),
    "Nearest Neighbors": KNeighborsClassifier(),
    "Linear SVM": SVC(),
    "Gradient Boosting Classifier": GradientBoostingClassifier(),
    "Decision Tree": tree.DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(n_estimators=18),
    "Neural Net": MLPClassifier(alpha=1),
    "Naive Bayes": GaussianNB()
}
```

```
In [30]: # Use Cross-validation.
from sklearn.model_selection import cross_val_score

# Logistic Regression
log_reg = LogisticRegression()
log_scores = cross_val_score(log_reg, X_train, y_train, cv=3)
log_reg_mean = log_scores.mean()
```

```

# SVC
svc_clf = SVC()
svc_scores = cross_val_score(svc_clf, X_train, y_train, cv=3)
svc_mean = svc_scores.mean()

# KNearestNeighbors
knn_clf = KNeighborsClassifier()
knn_scores = cross_val_score(knn_clf, X_train, y_train, cv=3)
knn_mean = knn_scores.mean()

# Decision Tree
tree_clf = tree.DecisionTreeClassifier()
tree_scores = cross_val_score(tree_clf, X_train, y_train, cv=3)
tree_mean = tree_scores.mean()

# Gradient Boosting Classifier
grad_clf = GradientBoostingClassifier()
grad_scores = cross_val_score(grad_clf, X_train, y_train, cv=3)
grad_mean = grad_scores.mean()

# Random Forest Classifier
rand_clf = RandomForestClassifier(n_estimators=18)
rand_scores = cross_val_score(rand_clf, X_train, y_train, cv=3)
rand_mean = rand_scores.mean()

# NeuralNet Classifier
neural_clf = MLPClassifier(alpha=1, max_iter=500, random_state=42)
neural_scores = cross_val_score(neural_clf, X_train, y_train, cv=3)
neural_mean = neural_scores.mean()

# Naives Bayes
nav_clf = GaussianNB()
nav_scores = cross_val_score(nav_clf, X_train, y_train, cv=3)
nav_mean = nav_scores.mean()

# Create a Dataframe with the results.
d = {'Classifiers': ['Logistic Reg.', 'SVC', 'KNN', 'Dec Tree', 'Grad B CLF', 'Rand FC', 'Neural Classifier', 'Naives Bayes'],
     'Crossval Mean Scores': [log_reg_mean, svc_mean, knn_mean, tree_mean, grad_mean, rand_mean, neural_mean, nav_mean]}

result_df = pd.DataFrame(data=d)

```

```

In [31]: # All our models perform well but I will go with GradientBoosting.
result_df = result_df.sort_values(by=['Crossval Mean Scores'], ascending=False)
result_df

```

```

Out[31]:

```

	Classifiers	Crossval Mean Scores
5	Rand FC	0.997494
3	Dec Tree	0.993758
4	Grad B CLF	0.991252
6	Neural Classifier	0.983756
7	Naives Bayes	0.983756
0	Logistic Reg.	0.982503
2	KNN	0.807477
1	SVC	0.796251

```

In [32]: # Cross validate our Gradient Boosting Classifier
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(rand_clf, X_train, y_train, cv=3)

```

```

In [33]: from sklearn.metrics import accuracy_score
rand_clf.fit(X_train, y_train)
y_prob = rand_clf.predict_proba(X_test)[: , 1]
print ("Random Forest is %2.2f" % accuracy_score(y_train, y_train_pred))

```

Random Forest is 0.99

```

In [34]: # Let's find the scores for precision and recall.
from sklearn.metrics import precision_score, recall_score
# The model is 77% sure that the potential client will suscribe to a term deposit.
# The model is only retaining 60% of clients that agree to suscribe a term deposit.
print('Precision Score: ', precision_score(y_train, y_train_pred))
# The classifier only detects 60% of potential clients that will suscribe to a term deposit.
print('Recall Score: ', recall_score(y_train, y_train_pred))
from sklearn.metrics import f1_score
f1 = f1_score(y_train, y_train_pred)
print(f'F1 Score = {f1}')

```

Precision Score: 0.9968304278922345
Recall Score: 0.9921135646687698
F1 Score = 0.9944664031620554

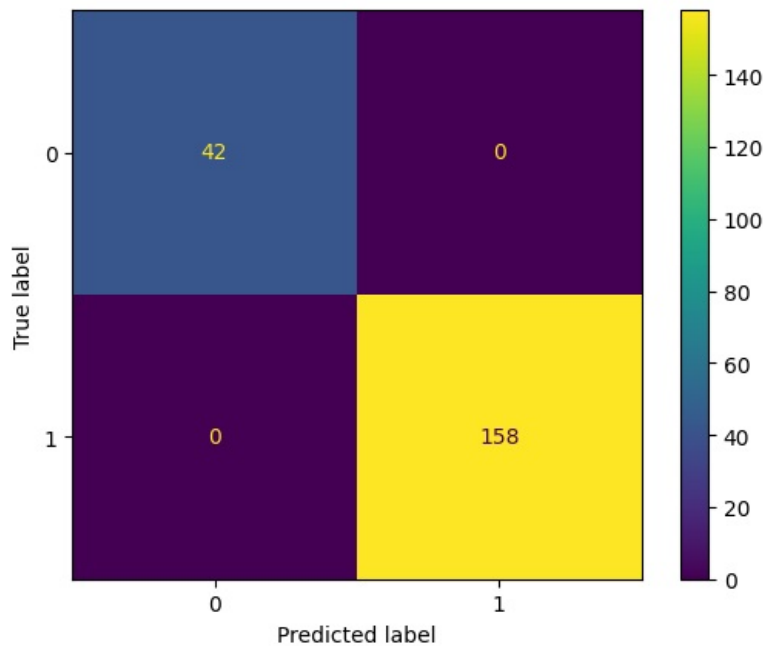
MATRIZ DE CONFUSION

```
In [35]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
# Predicciones
y_pred = rand_clf.fit(X_train, y_train).predict(X_test)

# Matriz de confusión
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
```

```
Out[35]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x16a41014980>
```



ROC CURVE

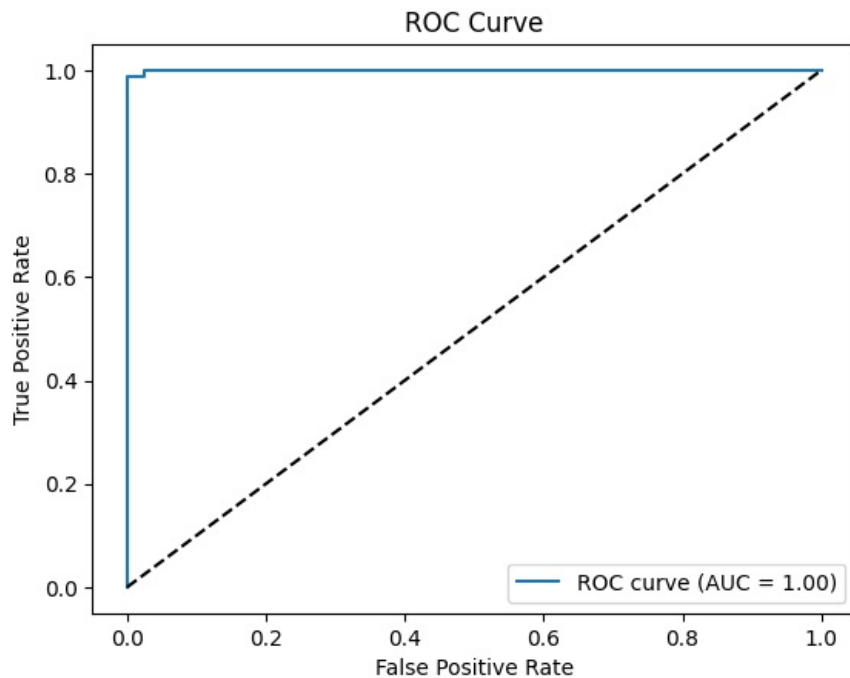
ROC CURVE RANDOM FOREST

```
In [36]: from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt
```

```
# Entrenar y predecir
neural_clf.fit(X_train, y_train)
y_prob = neural_clf.predict_proba(X_test)[: , 1]

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob, pos_label=1.0)
roc_auc = auc(fpr, tpr)

# Graficar
plt.figure()
plt.plot(fpr, tpr, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc='lower right')
plt.show()
```



ROC CURVE RANDOM FOREST Y MLPCLASSIFIER

```
In [37]: from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Entrenar modelos
rf_clf = RandomForestClassifier(random_state=42)
nn_clf = MLPClassifier(alpha=1, max_iter=500, random_state=42)

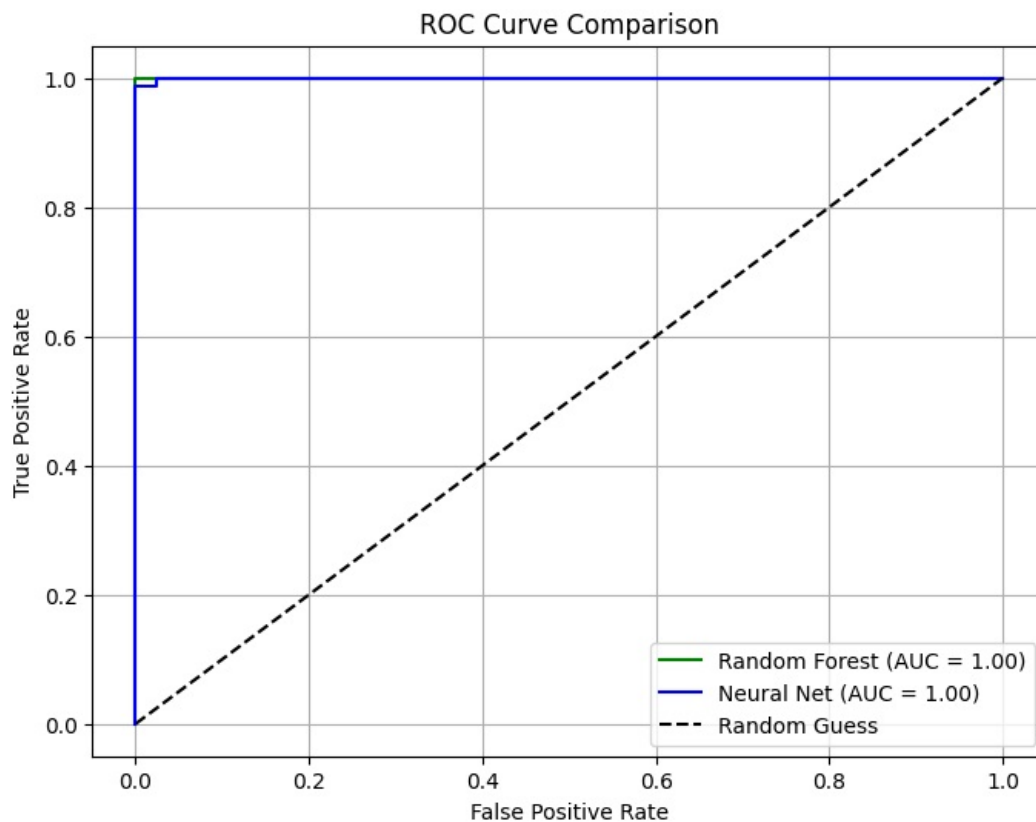
rf_clf.fit(X_train, y_train)
nn_clf.fit(X_train, y_train)

# Obtener probabilidades
rf_probs = rf_clf.predict_proba(X_test)[: , 1]
nn_probs = nn_clf.predict_proba(X_test)[: , 1]

# Calcular curvas ROC
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_probs, pos_label=1.0)
fpr_nn, tpr_nn, _ = roc_curve(y_test, nn_probs, pos_label=1.0)

# Calcular AUC
auc_rf = auc(fpr_rf, tpr_rf)
auc_nn = auc(fpr_nn, tpr_nn)

# Graficar
plt.figure(figsize=(8, 6))
plt.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {auc_rf:.2f})', color='green')
plt.plot(fpr_nn, tpr_nn, label=f'Neural Net (AUC = {auc_nn:.2f})', color='blue')
plt.plot([0, 1], [0, 1], 'k--', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



```
In [38]: from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

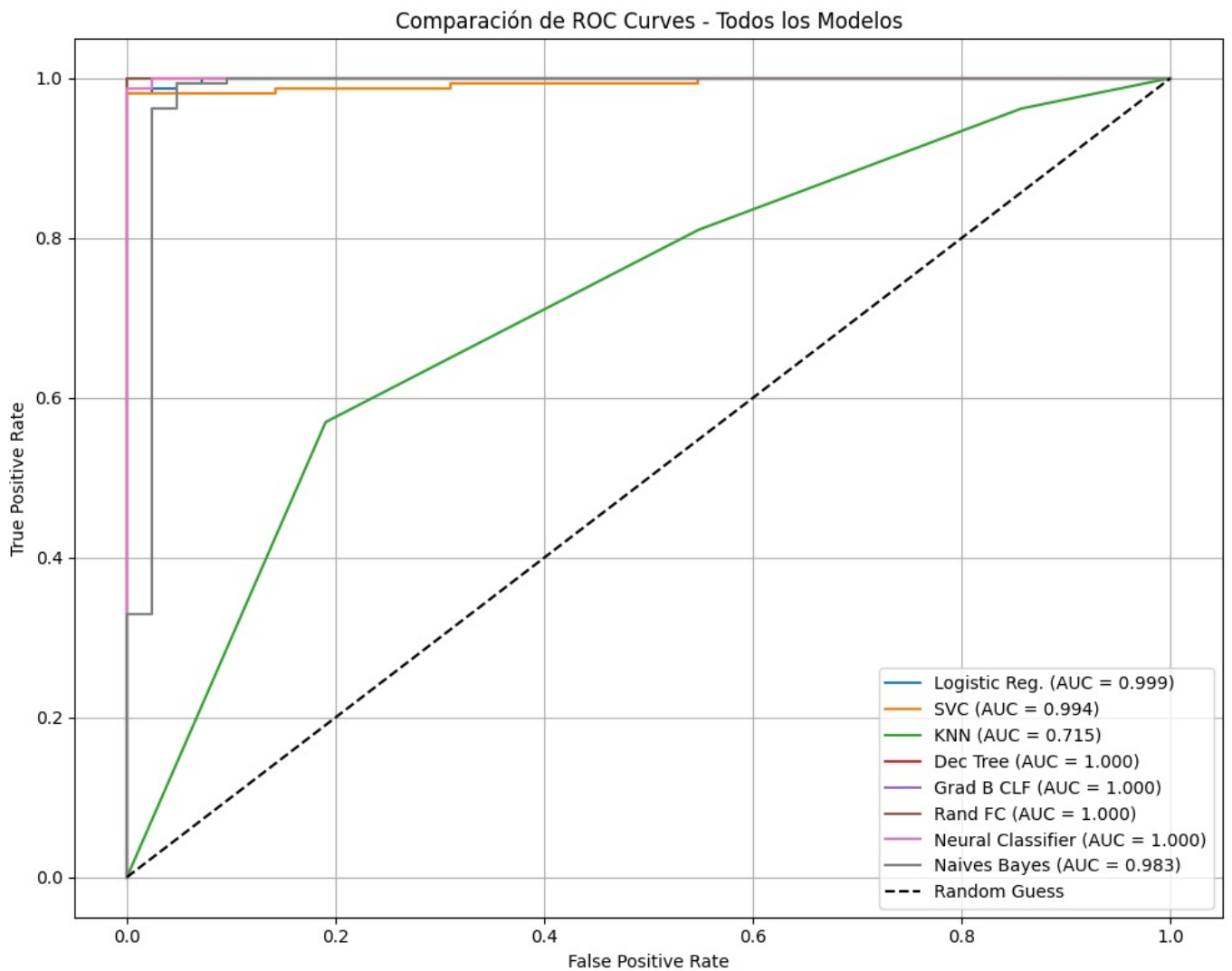
# Asegúrate de que SVC tenga probability=True
svc_clf = SVC(probability=True)

# Diccionario de modelos
models = {
    'Logistic Reg.': LogisticRegression(),
    'SVC': svc_clf,
    'KNN': KNeighborsClassifier(),
    'Dec Tree': tree.DecisionTreeClassifier(),
    'Grad B CLF': GradientBoostingClassifier(),
    'Rand FC': RandomForestClassifier(n_estimators=18),
    'Neural Classifier': MLPClassifier(alpha=1, max_iter=500, random_state=42),
    'Naives Bayes': GaussianNB()
}

# Graficar ROC curves
plt.figure(figsize=(10, 8))

for name, model in models.items():
    model.fit(X_train, y_train)
    y_prob = model.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob, pos_label=1.0)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc:.3f})')

# Línea base
plt.plot([0, 1], [0, 1], 'k--', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Comparación de ROC Curves - Todos los Modelos')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()
```



VARIABLES CLAVES

```
In [40]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

# Entrenar el modelo
modelo_rf = RandomForestClassifier(n_estimators=100, random_state=42)
modelo_rf.fit(X, y)

# Obtener la importancia de las variables
importancias = modelo_rf.feature_importances_

# Crear DataFrame con las importancias
ranking_importancia = pd.DataFrame({
    'Variable': X.columns,
    'Importancia': importancias
}).sort_values(by='Importancia', ascending=False)

# Mostrar las 15 más importantes
top_variables = ranking_importancia.head(15)

# Gráfico
plt.figure(figsize=(10, 6))
ax = sns.barplot(x='Importancia', y='Variable', data=top_variables, palette='viridis')
ax.set_title('Top 15 Variables más Importantes - Random Forest')
ax.set_xlabel('Importancia')
ax.set_ylabel('Variable')
plt.tight_layout()
plt.show()
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

